

DATA STRUCTURES

① $top = -1 \rightarrow$ empty Stack

$top = top + 1$ ← increment by one.

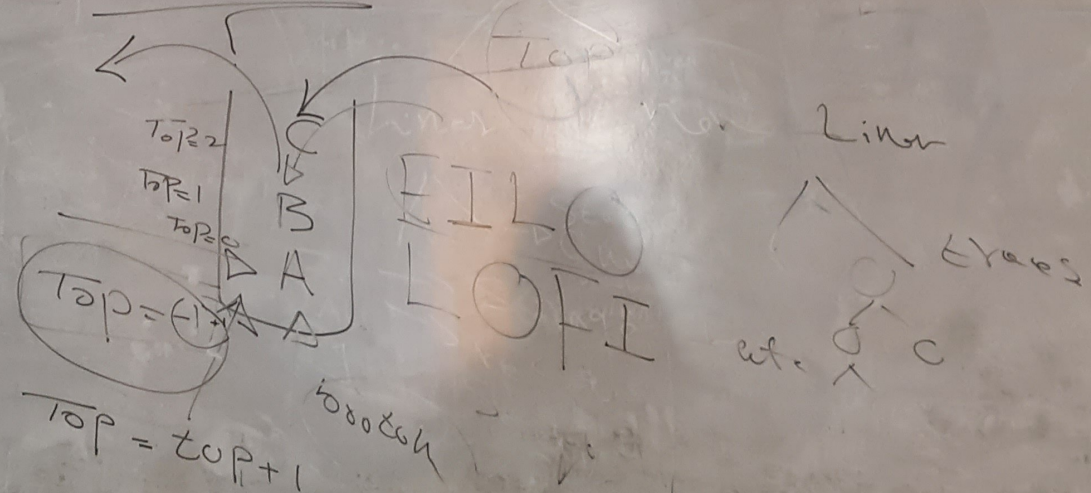
→ Full - max size

Stack

② Insertion $\rightarrow$ Push()

③ Access $\rightarrow$ top

④ Size()



HW

DATA STRUCTURES

① $top = -1 \rightarrow$ empty Stack

$top = top + 1$ ← increment by one

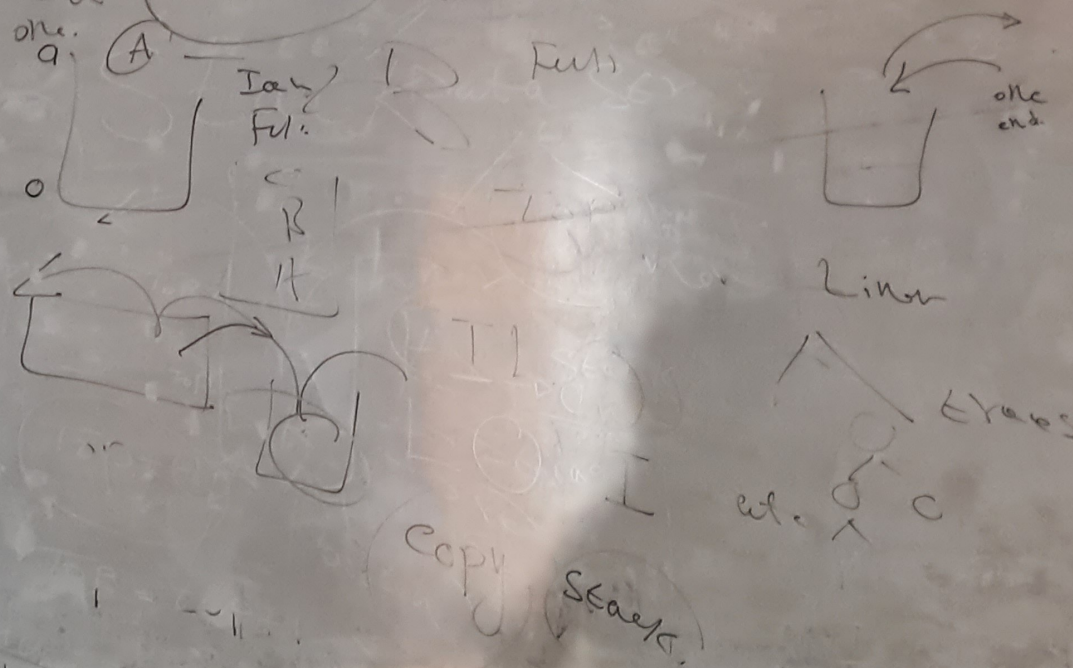
Size = 10

full - max size

② Insertion $\rightarrow$ Push()

③ Access $\rightarrow$ Pop()

④ Size()



DATA STRUCTURES

```
// include <Stack>
// top = 0
```

Full - max Size

For - (i/n)

Stack < data type >

Stack < int > myStack

Stack < int > Ali

Stack < String > Saad

Pop()

2 | C
1 | B
0 | A

Stack

Push(A)

Pop(C)

Pop(B)

Pop(A)

TOP = -1

AIDS

Push

Pop

Errors

Stack

by

10

DATA STRUCTURES

Push() -

Pop() -

top - C

Size - C

isEmpty() -

Push()
top++

Full - max Size

For - (n-1)

2 | C
1 | B
0 | A

hasa

Shada

Push(A)
= (B)

one end

Push

Pop

top = 2 Pop(C)

top = 1 Pop(B)

top = 0 Pop(A)

top = -1

Evans

decisions

by

10

Push()
top++

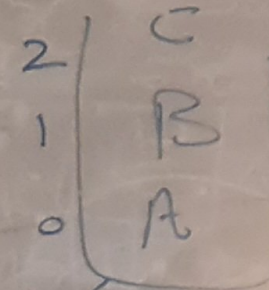
Full - max Size

If $top = size - 1$ then $i++$

Stack 'overflow'

$top = top + 1$

Stack(top) = X



$top = 0$, Push
 $top = 1$, Push
 $top = 2$, Push

hasa

Shada

$top = 2$

$top = 1$

$top = 0$

$top = -1$

$top = -2$

$top = -3$

$top = -4$

$top = -5$

$top = -6$

$top = -7$

DATA STRUCTURES

```

#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> myStack;
    myStack.push(10);
    myStack.push(20);
    myStack.push(30);
    cout << myStack.top();
    return 0;
}

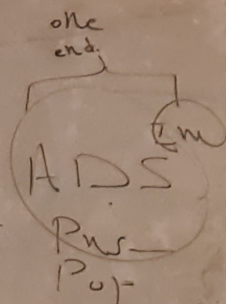
```

Push()
top()

Full - max size For - 1

2 | C
1 | B
0 | A

Push(A)
= (B)
= (C)



Shada. top
Pop(C)
Pop(B)
Pop(A)
top = -1

decrease by 1

DATA STRUCTURES

```

#include <iostream>
#include <stack> using namespace std;

int main() {
    stack<int> myStack;
    myStack.push(10);
    myStack.push(20);
    myStack.push(30);

    cout << "the top of stack is: " << myStack.top();
    myStack.pop();
    cout << "the top after popping is: " << myStack.top();

    return 0;
}
    
```

Push() top(x)

Full size

For (in)

2 C
1 B
0 A

-hasa

Stack top

Push(A)
top = 0
Push(B)
top = 1
Push(C)
top = 2

Pop(C)
top = top - 1
Pop(B)
top = top - 1
Pop(A)
top = -1

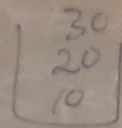
one end
AIDS
Push
Pop

Excess
by
10

Technical paper 10%
10/10/10
Access
Joining S
by "entire"

DATA STRUCTURES

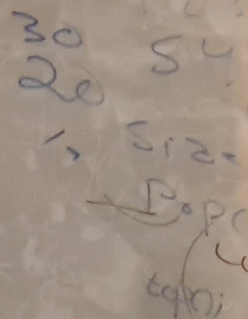
Push()



Stack

```

myStack.Pop();
while(!myStack.empty()) {
    cout << myStack.top();
    myStack.Pop();
    return 0;
}
    
```



Size

for (i=1

AIDS

Pop(C)
 top = top - 1
 Pop(R)
 top = top - 1
 Pop(A)
 top = -1

decrement by 1

technology
 نوآوری و فناوری

مع آگاهی از این امر
 201/6/16

Process of
 joining similar
 by end of line

DATA STRUCTURES

Push()

30
20
10

Stack

myStack.pop();

while(!myStack.empty()) {

cout << myStack.top();

myStack.pop();

return 0; }

30
20
10

Size

Pop()

top()

top()

top()

Size

for (i=0; i<n; i++)

one end

AIDS

Push

Pop

pop()

pop()

pop()

pop()

pop()

pop()

pop()

pop()

pop()

pop()

pop()

pop()

pop()

pop()

pop()